

Tema 8: Punters i memòria dinàmica

1. La memòria en C++: Stack vs Heap

Per entendre els punters, primer hem de saber on s'emmagatzemen les dades. La memòria d'un programa es divideix principalment en dues zones:

Característica	Stack (Pila)	Heap (Monticle)
Gestió	Automàtica (per l'ordinador).	Manual (pel programador).
Velocitat	Molt ràpida.	Més lenta.
Mida	Limitada i fixa.	Molt gran (memòria RAM disponible).
Cicle de vida	Lligat a les claus {} (stack frames).	Decidim quan neixen (new) i moren (delete).

Exemple de scope (Pila):

```
int f(int a, int b) {  
    int n = a;  
    if (b > n) {  
        int m = 2; // Neix aquí  
        a = b;  
    } // m Mor aquí automàticament  
    return a;  
} // n i a Moren aquí
```

2. Què és un punter?

Un **punter** és una variable que, en lloc d'emmagatzemar un valor (com un `int` o `char`), emmagatzema una **adreça de memòria**.

Operador	Nom	Funció	Exemple
<code>&</code>	Adreça de	Obté l'adreça de memòria d'una variable.	<code>p = &x;</code>
<code>*</code>	Desreferència	Accedeix al contingut de l'adreça que guarda el punter.	<code>cout << *p;</code>
<code>-></code>	Fletxa	Accés a membre via punter. Equivalent a <code>(*p).membre</code> .	<code>pp->first = "b";</code>

Trampes de declaració i sintaxi

- **Declaració múltiple:** L'asterisc `*` s'ha de posar per cada variable.

```
int *pb, *pc; // Dos punters
int* pb, pc; // pb és punter, pc és un ENTER normal! (Error típic)
```

- Punter a elements de contenidors:

```
vector<int> v = {1, 2, 3};
int *p = &v[1]; // Apunta al '2'
*p += 1;        // v[1] ara és 3
```

- Punter a membres de pair o struct:

```
pair<string, int> a = {"a", 7};
int *pi = &a.second;
*pi = 0; // a.second ara és 0
```

nullptr vs Inicialització: - `int *p = nullptr;` -> El punter apunta a “res”. Segur. - `int *p;` -> El punter apunta a una **adreça aleatòria** (brossa). Molt perillós. - `int *p = 5;` -> **ERROR**. Estàs dient que l’adreça de memòria es la número 5. Això provocarà un **SEGFault** segur.

```
int x = 10;
int* p = &x; // p apunta a x

cout << p;    // Imprimeix una adreça: 0x7ffe...
cout << *p;   // Imprimeix el valor de x: 10
```

3. Gestió dinàmica de memòria

Aquesta és la utilitat real dels punters: demanar memòria en temps d’execució.

Objectes vs vectors dinàmics

Podem demanar un sol objecte o un bloc sencer (vector) al Heap usant **new**, i alliberar-lo amb **delete**.

```
// Objectes simples
Data *pd = new Data(2025, 4, 2);
pair<int, int> *pp = new pair<int, int>(1, 2);
```

```
// Vectors dinàmics (Molt comú a C)
int *pv = new int[100];
char *pc = new char[100000];
```

Memory Leak: Es produeix quan perds l’adreça i ja no pots fer **delete**.

```
int *p = new int[100];
p = new int[100]; // ERROR: S'ha perdut l'adreça del primer vector! Fuga de memòria.
```

4. Aliasing i assignació

L’**aliasing** passa quan dos o més punters apunten a la mateixa adreça de memòria. Modificar el valor a través d’un punter afecta a tots els altres “àlies”.

```
int x = 10;
int* p1 = &x;
int* p2 = p1; // Aliasing: p2 apunta on apunta p1

*p2 = 20;
cout << x; // Imprimirà 20!
```

Exemple avançat: Un vector de punters apuntant al mateix objecte.

```
int x = 3;
vector<int*> v(10, &x); // 10 punters que apunten TOTS a la x

for (int i = 0; i < v.size(); ++i) {
    (*v[i])++; // Incrementem x 10 vegades!
}
cout << x; // Imprimirà 13
```

5. El perill dels punters: errors comuns

L'ús de punters requereix molta disciplina. Els errors més habituals a PRO2 són:

1. **Segmentation Fault (SEGFault)**: Intentar accedir a una adreça que no et pertany.
 - Desreferenciar nullptr: `int *p = nullptr; *p = 5;`
 - Accés fora de rang en vectors: `vector<int> v; v[13] = 0;`
2. **Memory Leak**: Destruir l'única referència a una memòria dinàmica sense alliberar-la.
3. **Dangling Pointer (Punter penjant)**: Punter que apunta a una adreça que ja ha estat alliberada amb `delete`.
4. **Double-Delete**: Fer `delete` dues vegades sobre la mateixa adreça (corromp el heap).

Al Makefile, utilitza el flag `-D_GLIBCXX_DEBUG`. Això activa comprovacions de seguretat en els contenidors de la STL i t'avisarà dels accessos fora de rang en lloc de donar-te un SEGFault silent o dades brossa.

Operació	Iterador (STL)	Punter (Baix Nivell)
Inici	<code>auto it = v.begin();</code>	<code>int *px = &x;</code>
Accés	<code>*it = 5;</code>	<code>*px = 5;</code>
Avançar	<code>it++;</code>	<code>px++;</code> (Avança una adreça)
Reassignar	<code>it = v.erase(it);</code>	<code>px = &y;</code>

Un punter pot ser vist com un iterador d'un vector, però un iterador d'un `std::list` no és necessàriament un punter (internament pot ser més complex).

6. Pas de paràmetres

Tipus	Sintaxi	Efecte	Eficiència
Per valor	<code>f(int x)</code>	Còpia del valor.	Baixa (si l'objecte és gran).
Per referència	<code>f(int& x)</code>	Àlies directe.	Alta.
Per punter	<code>f(int* pi)</code>	Passa l'adreça. Més ràpid que per valor.	

Exemple d'increment:

```
void inc(int *pi) {
    (*pi)++; // vol dir *pi += 1;
}

int i = 5;
inc(&i); // i ara val 6
```

A PRO2, preferim **referències constants** (`const T& x`) per a objectes grans que no volem modificar, i **pun-
ters** només quan necessitem que el paràmetre pugui ser opcional (`nullptr`) o per a estructures dinàmiques.

7. Aplicació: Estructures Enllaçades (Nodes)

La utilitat real dels punters a PRO2 és crear col·leccions de dades que creixen i decreixen node a node. Cada element s'emmagatzema en un **Node** (o **Item**).

El Node Bàsic

```
struct Item {
    T value;      // La dada
    Item* next;  // El "cable" cap al següent node
};
```

7.1 La Pila (Stack) - Model LIFO

En una pila enllaçada, només tenim un punter al cim (**ptopitem**). Cada vegada que fem **push**, el nou node apunta a l'antic cim.

::stackviz

[!TIP] **Exercici swap2Topmost()**: Per intercanviar els dos primers nodes sense tocar els **.value**, has de: 1. Guardar el segon node en un punter auxiliar: `Item *p2 = ptopitem->next;` 2. Reenllaçar: `ptopitem->next = p2->next;` 3. Fer que el nou primer sigui el que era segon: `p2->next = ptopitem;` 4. Actualitzar el cim: `ptopitem = p2;`

7.2 La Cua (Queue) - Model FIFO

En una cua, necessitem dos punters: **first** (per treure) i **last** (per afegir).

::queueviz

[!IMPORTANT] **Exercici operator[]**: Com que una cua enllaçada no és un vector, per trobar l'element **i** has de fer un bucle **for** que avanci el punter `p = p->next` exactament **i** vegades començant des de **first**.

7.3 Com esborrar nodes pel mig

Per esborrar un node (com a **removeFirstOccurrence**), necessites "saltar-te'l": 1. Trobar el node **anterior** al que vols esborrar (**ant**). 2. Fer el salt: `ant->next = ant->next->next;` 3. Alliberar la memòria: `delete p_a_esborrar;`

::pointerviz

Checklist per al Jutge (Punters)

- ☐ **Has posat nullptr?** Comprova sempre si un punter és nul abans de fer `p->next`.
- ☐ **Has fet delete?** Cada `new` ha de tenir el seu `delete` per evitar Memory Leaks.
- ☐ **Casos buits:** Què fa el teu codi si la pila/cua està buida? I si té només 1 element?
- ☐ **Auto-assignació:** En l'ús de `operator=`, has comprovat `if (this != &s)?`